

Programming Problems: OpenMP, MPI, and Hybrid MPI+OpenMP

Language: C++

Section 1: OpenMP

1. Parallel Array Sum & Statistics

Write a program that computes the sum, mean, and standard deviation of a large array ($\geq 10^7$ elements) using `#pragma omp parallel` for with a reduction clause. Compare execution time and speedup against the sequential version for 1, 2, 4, and 8 threads.

2. Parallel Matrix Multiplication

Implement dense matrix-matrix multiplication ($C = A \times B$) using OpenMP, parallelizing the outer loop over rows. Experiment with static vs. dynamic scheduling and report which performs better and why, given cache effects.

3. Parallel Prime Sieve (Sieve of Eratosthenes)

Parallelize the Sieve of Eratosthenes to find all primes up to N ($N \geq 10^8$). Discuss the load-balancing challenge caused by unequal marking cost across the range and compare static, dynamic, and guided scheduling.

4. Producer-Consumer with OpenMP Tasks

Implement a producer-consumer pipeline using OpenMP tasks (`#pragma omp task`) where producer tasks generate work items into a shared bounded buffer and consumer tasks process them. Use appropriate synchronization (critical/atomic/locks) to avoid race conditions.

5. Parallel Merge Sort with Tasks

Implement merge sort using OpenMP tasks for the recursive divide step (task/taskwait), with a sequential cutoff for small subarrays. Measure speedup for varying array sizes and cutoff thresholds, and plot the effect of thread count on performance.

Section 2: MPI

1. Distributed Array Sum

Distribute a large array of numbers evenly across MPI processes using `MPI_Scatter`, compute each process's partial sum, and combine results with `MPI_Reduce`. Handle the case where the array size is not evenly divisible by the number of processes.

2. Parallel Matrix-Vector Multiplication

Distribute the rows of a matrix across MPI processes (block decomposition). Each process computes its partial matrix-vector product and results are collected at rank 0 using `MPI_Gather`. Extend to handle non-square, non-evenly-divisible matrices.

3. Ring-Based Message Passing

Implement a program where each MPI process sends a message to its right neighbor and receives from its left neighbor in a ring topology, accumulating a running value as the message circulates. Use `MPI_Sendrecv` to avoid deadlock, and detect deadlock by first implementing it incorrectly with blocking `Send/Recv`.

4. Parallel Trapezoidal Rule for Numerical Integration

Estimate the definite integral of a function using the trapezoidal rule, dividing the interval among MPI processes. Each process computes its local sub-integral and results are combined with MPI_Reduce. Compare accuracy and runtime for varying numbers of processes and subintervals.

5. Parallel Sample Sort

Implement sample sort across MPI processes: each process sorts its local data chunk, a set of splitters is chosen and broadcast, data is redistributed with MPI_Alltoallv according to the splitters, and each process does a final local sort. Verify global sortedness of the combined output.

Section 3: Hybrid MPI + OpenMP (5 Problems)

1. Hybrid Vector/SAXPY Computation

Distribute a large vector operation ($y = a*x + y$) across MPI processes with block decomposition; within each process, parallelize the local loop with OpenMP. Measure and compare performance for different combinations of process count and thread count at a fixed total core count.

2. Hybrid Matrix-Matrix Multiplication

Distribute rows of matrix A across MPI processes; within each process, use OpenMP to parallelize the loop computing each process's block of the result matrix C. Compare against an MPI-only and an OpenMP-only version of the same problem for a fixed total number of cores.

3. Hybrid 2D Stencil / Heat Diffusion

Decompose a 2D grid into row-blocks across MPI processes, exchanging halo rows with neighbors via MPI_Sendrecv each iteration. Use OpenMP to parallelize the interior-point update within each process. Report the breakdown of time spent in communication vs. computation as grid size and process/thread counts vary.

4. Hybrid Monte Carlo Simulation

Estimate an integral or probability (e.g., estimating π) using Monte Carlo sampling. Each MPI process generates a local share of random samples, using OpenMP to parallelize sample generation across threads with thread-safe per-thread random number generators. Combine partial counts with MPI_Reduce.

5. Hybrid Parallel K-Means Clustering

Distribute data points across MPI processes. Within each process, use OpenMP to parallelize the assignment of points to the nearest centroid. Combine partial centroid sums and counts with MPI_Allreduce, then update and broadcast centroids until convergence. Compare convergence time against an MPI-only implementation.